Q-FLOW

Quantum-inspired metaheuristic optimisation
for intelligent traffic routing

Technical report · Smart India Hackathon problem SIH26137 · October 2026

Team: (add team name and members) **Platform:** web app + Android app (Capacitor) **Code:** qflow-v3

Abstract

Urban vehicle routing under congestion is an NP-hard combinatorial optimisation problem. Q-FLOW is a complete software platform that models a city road network as a weighted graph, with travel times from the Bureau of Public Roads (BPR) congestion model or from live TomTom traffic, and solves the capacitated vehicle routing problem with time windows (CVRPTW) using quantum-behaved particle swarm optimisation (QPSO), a quantum-inspired algorithm that runs on ordinary classical hardware. QPSO is combined with an optimal split decoder, local search and heuristic seeding, and is benchmarked under equal evaluation budgets on held-out, seeded instances against simulated annealing (SA), a genetic algorithm (GA), ant colony optimisation (ACO), classical PSO, the Clarke-Wright heuristic and an exact solver.

On held-out instances QPSO is within **0.1%** of the exact optimum at 8 customers (best on 9 of 10 instances), reaches a mean gap of **8.9%** against SA's 18.0% at 20 customers (clearly lower, more than two standard errors), ties SA on the mean at 50 customers while winning 3 of 5 instances, and has the lowest mean cost on 2 of 3 instances at 100 and 200 customers (preliminary). It is slower per run than SA in wall-clock time. The platform adds live-traffic routing, real-GPS turn-by-turn navigation with automatic rerouting, dynamic re-planning, and 20 automated test files. All results are reproducible with the commands in Section 10.

1. Problem and motivation

Indian cities face recurring congestion, inefficient route planning and high logistics costs. Delivery and service fleets must decide which vehicle visits which customers, in what order, within capacity limits and customer time windows, while travel times change with traffic. This is the vehicle routing problem (VRP), which is NP-hard: exact methods only scale to small instances, so practical systems use metaheuristics.

Quantum computers promise advantages for combinatorial optimisation, but current hardware cannot run problems of this size reliably. Quantum-inspired algorithms take ideas from quantum mechanics, here the delta-potential-well model behind QPSO, and run them on classical CPUs. SIH26137 asks for such a framework, modelled on a weighted graph, under real-time or simulated traffic, and benchmarked against conventional metaheuristics and exact methods.

2. System overview

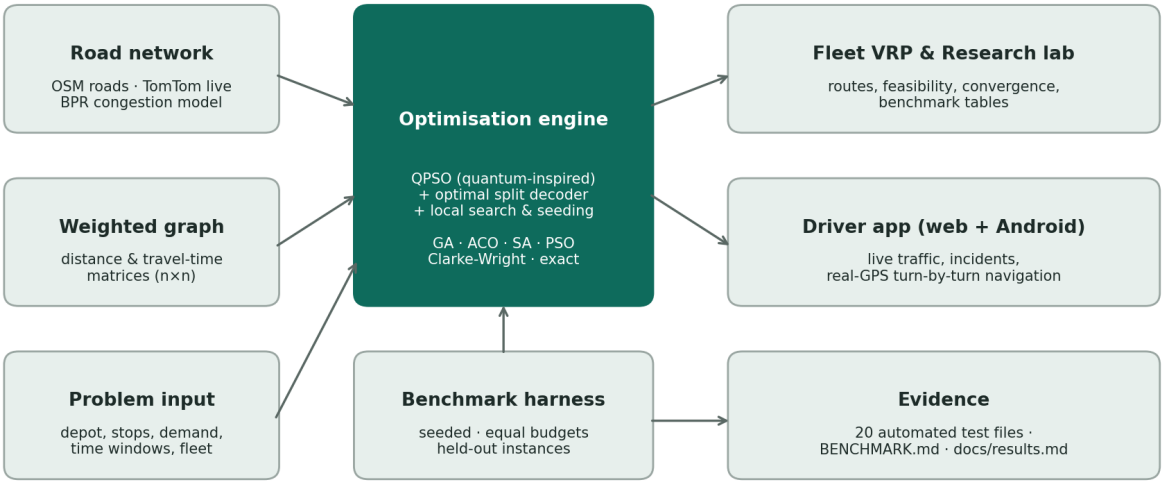


Figure 1. Q-FLOW architecture. All optimisation runs on the device (browser or phone); external services provide maps, place search, OSRM/TomTom routing and live traffic.

- **Optimisation engine** (src/vrp-engine.js, src/vrp-extras.js): QPSO, GA, ACO, SA, PSO, Clarke-Wright, exact solver, Dijkstra and QPSO shortest path, dynamic re-planning.
- **Fleet VRP and Research lab pages**: interactive instances on a map of Hyderabad, benchmark tables, feasibility flags and convergence charts.
- **Driver app**: route comparison with live TomTom travel times, live incidents, and real-GPS turn-by-turn navigation with off-route detection, rerouting, voice guidance and arrival detection (web and Android).
- **Evidence**: seeded benchmark script (npm run bench), extra experiments, and 20 automated test files (npm test).

3. Network model (weighted graph)

Nodes are the depot (0) and customers 1..n. Each ordered pair (i, j) is an edge with two weights: distance d_{ij} and travel time t_{ij}. Two traffic sources are supported:

- **Simulated (BPR):** $d_{ij} = 1.28 \times \text{haversine}(i, j)$ and $t_{ij} = t^0_{ij}\,(1 + 0.15\,(V/C)^4) + \delta_{ij}$, with free-flow speed 40 km/h, V/C the volume-to-capacity ratio (optionally varying by time of day and per road) and δ_{ij} an incident delay.

- **Live (TomTom):** d_{ij} and t_{ij} come from TomTom Matrix Routing with live traffic, on the real road network (verified with a working key: 9 of 9 matrix cells routed).

For the shortest-path problem the framework uses a weighted road graph with BPR edge costs, solved exactly by Dijkstra and heuristically by QPSO (Research lab §4).

4. Mathematical formulation (CVRPTW)

Each customer i has demand q_i, a time window $[e_i, l_i]$ and service time s_i; the fleet has K vehicles of capacity Q; the depot closes at l_0. A solution is a set of routes $r_1..r_m$, each starting and ending at the depot. The objective minimised by every algorithm is:

$$F = \Sigma_k D_k + 0.4\,\Sigma_k T_k + 1500\,\Sigma_k \max(0, L_k - Q) + 20\,\Sigma_i \max(0, a_i - l_i) + 10000\,\max(0, m - K)$$

D_k is the road distance of route k, T_k its completion time back at the depot (driving, waiting and service), L_k its load, and a_i the arrival time at customer i. Returning after the depot closes counts as lateness. Vehicles arriving early wait until e_i.

Constraint handling. Capacity, time windows and fleet size are soft constraints with large penalty weights, so every algorithm can move through infeasible regions; each result is then checked and flagged as feasible or infeasible, with the violated constraint reported (kg over capacity, minutes late, extra vehicles). Inputs are validated before solving (coordinates, demands, time windows, capacity, fleet size and traffic values), and impossible instances are diagnosed, e.g. a stop heavier than one vehicle or total demand above fleet capacity.

5. Algorithms

5.1 QPSO (quantum-behaved particle swarm optimisation)

Each particle encodes a customer order as random keys (one real number per customer; sorting gives the order). The position update is the Sun, Feng and Xu (2004) delta-potential-well rule:

$$p = \varphi \cdot pBest + (1 - \varphi) \cdot gBest, \quad x \leftarrow p \pm \alpha \cdot |mBest - x| \cdot \ln(1/u), \quad \varphi, u \sim U(0, 1)$$

mBest is the mean of all personal bests and α (contraction-expansion) falls from 0.95 to 0.50. Around this unchanged update, Q-FLOW adds:

- **Optimal split decoder** (splitTourSequence): dynamic programming cuts each customer order into at most K routes at the cheapest break points, using the same objective F. For the same order it is never worse than the greedy split (tested on 900 random orders: better in 81%, equal otherwise).
- **Lamarckian local search** with 2-opt, or-opt and swap moves (probability 0.25 per particle, up to 2n evaluations), plus an **elite local search** on the global best each iteration (up to 10n evaluations).
- **Heuristic seeding**: 15% of the initial swarm from Clarke-Wright, nearest-neighbour, Solomon time-oriented nearest-neighbour and earliest-due-date tours.
- All local-search evaluations count against the same budget as every other algorithm.

5.2 Baselines

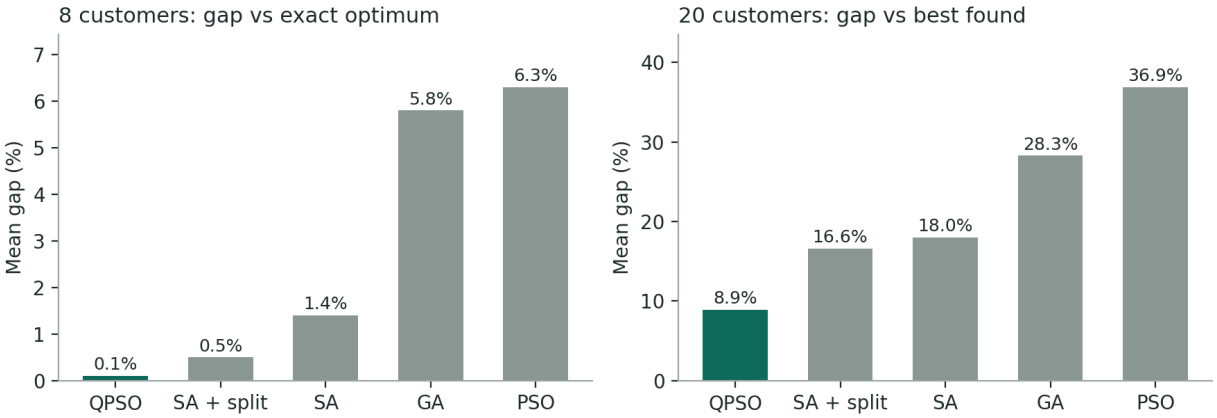
- **Simulated annealing**: 2-opt and swap moves, geometric cooling. Also run with the optimal split decoder as a fairness check.
- **Genetic algorithm**: order crossover (OX) and 2-opt mutation. **ACO**: pheromone and heuristic visibility. **Classical PSO**: random keys with inertia.
- **Clarke-Wright**: savings heuristic with a time-window check on each merge (one deterministic run).

- **Exact solver**: enumerates every customer order (n ≤ 9) and cuts each with the optimal split decoder. Any plan with at most K routes, joined route by route, is one of those orders, so this gives the true optimum of F for at most K vehicles.

6. Experimental method

- **Instances:** seeded random CVRPTW instances around Hyderabad (8, 20, 50 customers; 100 and 200 for scalability), vehicle capacity 80 kg, K = ⌈n/6⌉ vehicles (at least 2), BPR traffic level V/C = 1.4.
- **Equal budgets:** every stochastic algorithm gets the same number of objective evaluations, max(2000, min(20000, 300n)), fixed before any comparison.
- **Runs:** 10 independent seeded runs per instance (5 for the 100/200-customer study).
- **Tuning vs held-out:** QPSO settings were chosen on tuning seeds 101–110 and a separate capacity-tight tuning set (seeds 204, 205, 208–211). All results here use held-out seeds 1–10, never used for tuning.
- **Gap** = (mean cost − reference) / reference; the reference is the exact optimum at n = 8 and the best cost any method found otherwise. "Lowest mean cost" counts instances where the algorithm had the lowest mean among the metaheuristics.

7. Results



Held-out instances (seeds 1-10), 10 seeded runs each, equal evaluation budgets. Lower is better.

Figure 2. Mean gap on held-out instances (lower is better).

7.1 Eight customers (gap vs exact optimum)

Algorithm	Mean gap	Feasible runs	Lowest mean cost (instances)
QPSO (optimal split decoder)	0.1%	100%	9
SA + optimal split (fairness check)	0.5%	100%	1
QPSO, greedy decoder (ablation)	0.7%	100%	3
Simulated annealing (SA)	1.4%	99%	1
QPSO-core (ablation)	1.9%	97%	0
Genetic algorithm (GA)	5.8%	98%	0
Classical PSO	6.3%	100%	0
Ant colony (ACO)	260.5%	55%	0
Clarke-Wright (1 run)	848.3%	80%	0

QPSO is within 0.1% of the exact optimum and has the lowest mean cost on 9 of 10 instances. Paired by instance against SA with the same decoder, QPSO is lower on 9 of 10 (difference −0.4 points, standard error 0.1).

7.2 Twenty customers

Algorithm	Mean gap	Feasible runs	Lowest mean cost (instances)
QPSO (optimal split decoder)	8.9%	90%	8
QPSO, greedy decoder (ablation)	12.8%	90%	2
SA + optimal split (fairness check)	16.6%	88%	0
Simulated annealing (SA)	18.0%	89%	0
QPSO-core (ablation)	27.9%	80%	0
Genetic algorithm (GA)	28.3%	85%	0
Classical PSO	36.9%	70%	0
Ant colony (ACO)	87.3%	45%	0
Clarke-Wright (1 run)	168.2%	80%	3

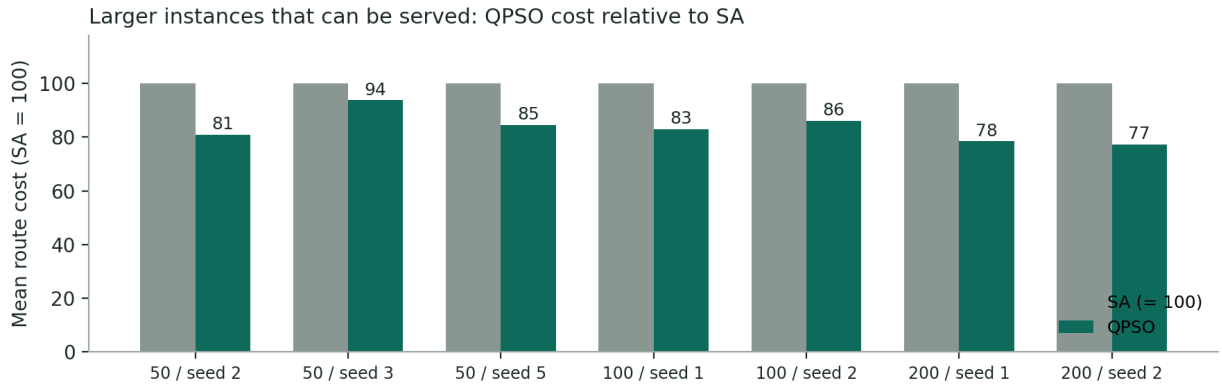
QPSO's mean gap is 8.9% against 18.0% for SA and 16.6% for SA with the same decoder; it is lower than SA-with-split on all 10 instances (−7.7 points, SE 1.6). The ablations show both parts matter: with the greedy decoder QPSO reaches 12.8%, and without local search and seeding 27.9%. The ranking is the same at half and double the budget.

7.3 Fifty customers

Algorithm	Mean gap	Feasible runs	Lowest mean cost (instances)
Simulated annealing (SA)	73.7%	58%	2
SA + optimal split (fairness check)	73.8%	60%	0
QPSO (optimal split decoder)	74.9%	58%	3
QPSO, greedy decoder (ablation)	79.9%	56%	0
Genetic algorithm (GA)	267.7%	26%	0
Clarke-Wright (1 run)	298.4%	40%	2
Ant colony (ACO)	347.8%	14%	0
Classical PSO	857.9%	0%	0
QPSO-core (ablation)	1,070.0%	0%	0

The mean gaps of QPSO (74.9%) and SA (73.7%) tie; QPSO has the lowest mean cost on 3 of 5 instances and a much lower median gap (4.0% vs 22.5%). The large means come from two instances whose total demand exceeds the fleet's capacity (101% and 104%), which no algorithm can serve; there SA leaves less overload. On the three serviceable instances QPSO's mean cost is lower than SA's on all three (1,137 vs 1,404; 1,459 vs 1,557; 1,165 vs 1,378). Only 5 instances: too few for a significance call.

7.4 Scalability (preliminary)



Customers / instance seed. 50: 10 runs each; 100 and 200: 5 runs each (preliminary). Over-capacity instances excluded here; see full tables.

Figure 3. Mean cost of QPSO relative to SA on instances that can be served, from 50 to 200 customers.

n	K	Runs	QPSO lowest mean	QPSO cost vs SA (serviceable)	Feasible QPSO / SA	Time per run QPSO / SA
100	17	3 × 5 runs	2 of 3	-17%, -14%	67% / 60%	3.2 s / 0.09 s
200	34	3 × 5 runs	2 of 3	-22%, -23%	67% / 13%	12.4 s / 0.27 s

At 100 and 200 customers QPSO has the lowest mean cost on 2 of 3 instances, 14–23% below SA. On the third instance at each size (104% and 99.9% fleet use) SA with the optimal split decoder is better. Three instances per size are too few for firm conclusions.

8. Convergence analysis

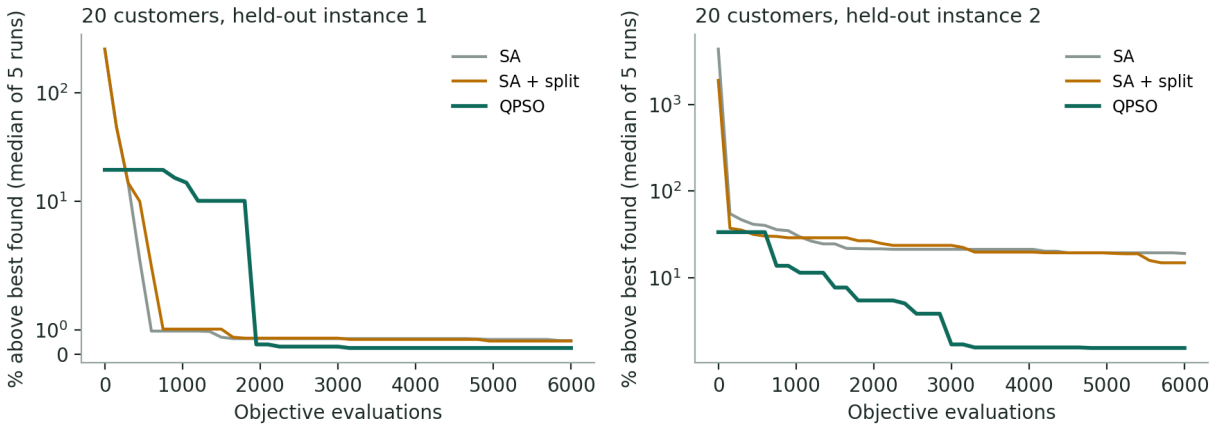


Figure 4. Convergence on two held-out 20-customer instances (median of 5 runs). Cost is shown as % above the best solution found by any run.

Algorithm (20 customers, 10 instances × 5 runs)	Runs within 5% of best	Median evaluations to 5%	Runs within 1% of best	Median evaluations to 1%	Time per run
QPSO (optimal split decoder)	68%	1,760	34%	3,120	104 ms
Simulated annealing	18%	1,481	12%	1,521	11 ms
SA + optimal split	12%	1,481	10%	1,521	97 ms

QPSO reaches near-best solutions far more often (68% of runs within 5%, against 18% for SA). SA converges early but stalls in local optima; the runs of SA that do reach the target get there in fewer evaluations. With 3,000 evaluations QPSO (10.2% mean gap) is already better than SA with 12,000 (15.3%).

Computational cost

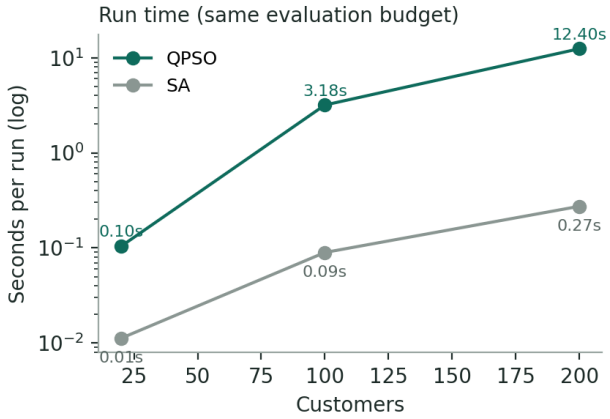


Figure 5. Wall-clock time per run with the same evaluation budget. The optimal split decoder makes each QPSO evaluation more expensive, so QPSO is 10–45× slower per run than SA (under 0.2 s at 20 customers, about 12 s at 200). QPSO's advantage is solution quality per evaluation, not wall-clock speed.

9. Platform features

- **Live traffic (TomTom)**: traffic flow on maps, real incidents, live travel times and a live travel-time matrix for the fleet optimiser. Verified with a working key (6 of 6 service checks, e.g. 253 incidents parsed near Hyderabad). Without a key, all traffic is the labelled BPR simulation.
- **Real-GPS navigation**: continuous device location, route progress, next maneuver, ETA, heading-up camera, off-route detection with automatic rerouting from the latest position, voice guidance and arrival detection.

- **Dynamic re-planning**: QPSO warm-starts from the current plan when conditions change, so the new plan is never worse than keeping the old one.
- **Honest labelling**: every number in the app is marked as live, estimated or simulated.
- **Quality**: 20 automated test files (engine, constraints, navigation, live traffic with mocked responses, UI flows), input validation and offline-safe Android build.

10. Reproducibility

Command	What it does
npm test	Runs all 20 test files
npm run bench	Re-creates the main benchmark (docs/results.md); deterministic
node scripts/extra-experiments.js	Convergence-speed and 100/200-customer runs (resumable; repeat until "done 60/60")
npm run tomtom:check	Checks a TomTom key against the six live services
npm run android:apk	Builds the Android app

11. Limitations

- QPSO is slower per run than SA (Section 8). An equal-time comparison has not been done.
- At 50+ customers QPSO is behind SA on the most capacity-tight and over-capacity instances; the 100/200-customer results use only 3 instances each.
- The exact solver is limited to 9 customers. No MILP solver (e.g. OR-Tools) has been run on larger instances.
- The VRP lab uses straight-line × 1.28 distances unless live TomTom traffic is switched on. The shortest-path experiment uses a synthetic road grid.
- Network-wide congestion is reduced only through each route's cost; the system-level traffic assignment page is a labelled simulation.
- Benchmark instances are synthetic; real fleet data has not been tested.

12. Conclusion and next steps

Q-FLOW delivers the platform requested by SIH26137: a weighted-graph network model with simulated and live traffic, a formal CVRPTW formulation with explicit constraint handling, a quantum-inspired QPSO solver, systematic and reproducible benchmarking against five metaheuristics and an exact method, and convergence and scalability analysis. On held-out instances QPSO matches the exact optimum closely at small sizes and gives the best routes per evaluation at medium sizes, with honest limits at large, capacity-tight sizes and in run time.

- Speed up the split decoder (incremental evaluation, Web Workers) to close the run-time gap.
- Benchmark 100–500 customers on more instances and against OR-Tools.
- Add an adaptive capacity penalty for over-capacity fleets.
- Pilot with real delivery data from a Hyderabad fleet.

References

Sun, J., Feng, B. and Xu, W. (2004). Particle swarm optimization with particles having quantum behavior. Proc. IEEE Congress on Evolutionary Computation.

Prins, C. (2004). A simple and effective evolutionary algorithm for the vehicle routing problem. Computers & Operations Research 31(12).

Clarke, G. and Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. Operations Research 12(4).

Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. Operations Research 35(2).

U.S. Bureau of Public Roads (1964). Traffic Assignment Manual.

Appendix A. Delivery table (expected deliverables)

Expected deliverable	Status	Where
Complete software platform	Done	Web app (npm run web) and Android APK
Quantum-inspired QPSO implementation	Done	src/vrp-engine.js (runQPSO)
Graph-based network modelling	Done	Section 3; distance/time matrices, BPR, TomTom live matrix
Mathematical formulation	Done	Section 4; Research lab §1
Constraint handling	Done	Section 4; feasibility flags, validation, tests
Convergence analysis	Done	Section 8; Fleet VRP and Research lab charts
Benchmark vs conventional metaheuristics	Done	Section 7; BENCHMARK.md, docs/results.md
Benchmark vs exact methods	Done (n ≤ 9)	Exact enumeration + optimal split; Dijkstra for shortest path
Real-time / simulated traffic	Done	TomTom live (verified) and BPR simulation
Scalability	Preliminary	Section 7.4; docs/extra-results.md
Technical report	Done	This document
Presentation	Done	Q-FLOW_SIH26137_Presentation.pptx
Demo video	Script ready	Appendix B (to be recorded)

Appendix B. Demo video script (about 2½ minutes)

0:00–0:15 Problem. City congestion; vehicle routing is NP-hard; why a quantum-inspired algorithm on classical hardware.

0:15–0:50 Fleet VRP. Choose 25 stops, run "QPSO vs metaheuristics". Show the routes on the map, the benchmark table, the feasibility flags and the convergence chart.

0:50–1:15 Research lab. Formulation, exact-optimum comparison, shortest path vs Dijkstra, dynamic re-planning.

1:15–1:40 Live traffic. Home map with TomTom traffic colours and incidents; Road alerts page.

1:40–2:20 Phone. Plan a trip, start navigation from real GPS, take a wrong turn, show automatic rerouting and arrival.

2:20–2:40 Results. QPSO 0.1% from optimum at 8 customers, 8.9% vs 18.0% at 20; limits at large, capacity-tight sizes and run time.